L08 • RoPE 旋转位置编码

> *"让模型知道谁先谁后"*

📋 本节目标

学完本节，你将能够：

1. 理解为什么 Transformer 需要额外的位置编码
2. 了解位置编码从绝对到相对、再到 RoPE 的发展历程
3. 掌握 RoPE 的核心数学原理（简化版）
4. 理解 rope_theta 参数和频率计算
5. 了解 YaRN 长度外推的基本思想
6. 读懂 MiniMind 中 `precompute_freqs_cis` 的源码

🔗 前置知识

- L07 · RMSNorm 归一化——了解 Transformer Block 的结构
- 三角函数基础：sin、cos 的含义
- 复数的基本概念（可选，不懂也能看懂核心内容）

---

1. 为什么需要位置编码？

1.1 Transformer 的"位置盲"问题

回忆一下 Self-Attention 的计算：每个 token 都和所有其他 token 计算注意力分数。这个过程是**完全对称的**——它不关心 token 出现在什么位置。

这意味着：

```
"猫 吃 鱼" 和 "鱼 吃 猫"
```

在没有位置编码的 Transformer 看来，这两句话**完全一样**！因为 Self-Attention 只看到了{猫, 吃, 鱼}三个 token 互相关注，不知道谁在前谁在后。

但显然，"猫吃鱼"和"鱼吃猫"意思完全不同。**位置编码的作用就是告诉模型每个 token 在序列中的位置。**

1.2 RNN 和 CNN 为什么不需要?

- **RNN**：天然按顺序处理，每个时间步依赖前一个时间步，位置信息隐含在计算顺序中
- **CNN**：卷积核有固定的感受野，相对位置信息编码在卷积核的权重中
- **Transformer**：所有位置并行计算，Self-Attention 是置换不变的（permutation invariant），必须显式注入位置信息

---

2. 位置编码的发展历程

2.1 绝对位置编码（Sinusoidal & Learned）

原始 Transformer（2017）使用正弦/余弦函数生成位置编码：

$$\text{PE}(pos, 2i) = \sin(\frac{pos}{10000^{2i/d}})$$

$$\text{PE}(pos, 2i+1) = \cos(\frac{pos}{10000^{2i/d}})$$

每个位置有一个固定的向量，直接**加到** Embedding 上：

$$\text{input} = \text{Embedding}(x) + \text{PE}(pos)$$

GPT 系列改用可学习的绝对位置编码：训练一个 (max_seq_len, d_model) 的参数矩阵。

绝对位置编码的问题

1. **无法外推**：如果训练时最大长度是 512，推理时无法处理长度 1024 的序列
2. **不能直接编码相对关系**："第 3 个词和第 5 个词的关系"需要模型自己学习

2.2 相对位置编码（ALiBi 等）

相对位置编码的核心思想：**不告诉模型"你在第几个位置"，而是告诉模型"两个 token 之间距离多远"。**

例如 ALiBi（Attention with Linear Biases）直接在注意力分数上减去一个与距离成正比的偏置。

2.3 RoPE：旋转位置编码（2021）

RoPE（Rotary Position Embedding）由苏剑林提出，是目前**最主流**的位置编码方案，被 LLaMA、Qwen、MiniMind 等模型采用。

RoPE 的优雅之处在于：它通过**旋转**来编码位置，使得两个位置的注意力分数**天然地只依赖于它们的相对距离**。

---

3. RoPE 的核心思想

3.1 一个直觉：用旋转角度表示位置

想象一个钟表的指针： - 位置 0 → 指针指向 12 点 - 位置 1 → 指针旋转 θ 度 - 位置 2 → 指针旋转 2θ 度 - 位置 m → 指针旋转 mθ 度

两个位置之间的"距离"就是它们的旋转角度之差，而这个差值**只取决于相对距离 (m-n)**，与绝对位置无关！

3.2 二维的情况：旋转矩阵

先从最简单的二维情况理解。假设我们有一个二维向量 $\mathbf{q} = (q_0, q_1)$，要给它加上位置 m 的信息。

RoPE 的做法是：**对这个向量施加一个旋转角度为 $m\theta$ 的旋转变换**：

$$R_m \mathbf{q} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} q_0 \\ q_1 \end{pmatrix}$$

结果：

$$R_m \mathbf{q} = \begin{pmatrix} q_0 \cos m\theta - q_1 \sin m\theta \\ q_0 \sin m\theta + q_1 \cos m\theta \end{pmatrix}$$

3.3 关键性质：内积只依赖相对位置

如果 Query 在位置 m，Key 在位置 n，它们的注意力分数是：

$$(R_m \mathbf{q})^T (R_n \mathbf{k}) = \mathbf{q}^T R_m^T R_n \mathbf{k} = \mathbf{q}^T R_{n-m} \mathbf{k}$$

这里用到了旋转矩阵的性质：$R_m^T R_n = R_{n-m}$

注意力分数只依赖相对距离 $n - m$，这就是 RoPE 的核心优势！

3.4 高维推广：两两分组

实际中，d_model 不是 2 维而是 768 维。RoPE 的处理方式是**将 768 维向量两两分组**，得到 384 组，每组是一个二维向量，分别施加不同频率的旋转：

```
维度 0,1 → 用频率 θ₀ 旋转
维度 2,3 → 用频率 θ₁ 旋转
维度 4,5 → 用频率 θ₂ 旋转
...
维度 766,767 → 用频率 θ₃₈₃ 旋转
```

每组的旋转频率不同，低维度组频率高（变化快），高维度组频率低（变化慢）。这类似于用不同周期的"时钟"来编码位置，就像用秒针、分针、时针来表示时间。

3.5 频率计算公式

$$\theta_i = \frac{1}{\text{base}^{2i/d}}$$

其中： - base 是 rope_theta 参数（MiniMind 使用 1e6） - i 是组的索引（0, 1, 2, ..., d/2-1） - d 是向量维度

对于位置 m，第 i 组的旋转角度为：

$$m \cdot \theta_i = \frac{m}{\text{base}^{2i/d}}$$

- 低维度（i 小）→ θ_i 大 → 旋转角度大 → 变化快（编码近距离关系）
- 高维度（i 大）→ θ_i 小 → 旋转角度小 → 变化慢（编码远距离关系）

---

4. rope_theta 参数的含义

4.1 rope_theta 是什么?

rope_theta（即公式中的 base）是 RoPE 的一个超参数，控制频率的分布。

模型	rope_theta
LLaMA 1	10,000
LLaMA 3	500,000
MiniMind	**1,000,000 (1e6)**

模型	rope_theta
Qwen2	1,000,000

4.2 rope_theta 的影响

rope_theta 越大： - 所有频率 θ_i 都会变小 - 旋转角度变化更慢 - 模型能处理的**有效序列长度更长**

直觉：想象一个时钟，如果秒针转得很慢，那么它能"计时"更长的时间而不会"绕回来"。

4.3 MiniMind 为什么用 1e6?

MiniMind 使用较大的 rope_theta = 1e6，这让模型天然具有更好的**长文本处理能力**。虽然 MiniMind 的训练序列长度可能不长，但这个参数为未来扩展留下了空间。

---

5. RoPE 的优点总结

优点	说明
天然的相对位置	注意力分数自动只依赖相对距离，无需额外设计
良好的外推能力	理论上可以处理比训练长度更长的序列
计算高效	只需要 sin/cos 和元素乘法，不需要额外参数
无额外参数	RoPE 不引入可学习参数，旋转频率是预计算的
长期衰减	距离越远的 token 对，注意力分数自然衰减

---

6. YaRN 长度外推

6.1 问题：模型能处理超长文本吗?

假设模型训练时的最大序列长度是 512 个 token，那么推理时能处理 4096 个 token 吗?

直接用 RoPE，超出训练长度的位置编码是"没见过"的，模型表现会急剧下降。这就是**长度外推（Length Extrapolation）**问题。

6.2 YaRN 的基本思想

YaRN（Yet another RoPE extensioN）是一种 RoPE 的外推增强方法。它的核心思想：

1. **频率插值**：把高频部分保持不变，低频部分做拉伸
2. **注意力缩放**：对注意力分数做适当缩放，补偿外推带来的分布偏移

直觉：想象你有一把只有 30cm 的尺子（训练长度 512），现在要量 2 米的东西（推理长度 4096）。YaRN 的做法不是简单地把刻度变稀（NTK 插值），而是： - 大的刻度（cm）变稀——对应低频拉伸 - 小的刻度（mm）保持不变——对应高频保持

6.3 MiniMind 中的 YaRN 支持

MiniMind 的配置中支持 YaRN 外推：

```
class LMConfig:
    rope_theta: float = 1e6          # 基础频率
    max_seq_len: int = 32768         # 支持的最大序列长度
    use_yarn: bool = False           # 是否启用 YaRN
```

---

7. MiniMind 源码解读

7.1 precompute_freqs_cis 函数

这是 RoPE 实现的核心函数，预计算所有位置的旋转角度：

```
def precompute_freqs_cis(dim, end, theta=1e6):
    # 计算每组的频率
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].float() / dim))
    # 生成位置序列
    t = torch.arange(end, device=freqs.device)
    # 外积：每个位置 × 每个频率 = 旋转角度矩阵
    freqs = torch.outer(t, freqs)
    # 用复数表示旋转（cos + i*sin）
    freqs_cis = torch.polar(torch.ones_like(freqs), freqs)
    return freqs_cis
```

7.2 逐行解析

第一行：计算频率

```
freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].float() / dim))
```

- `torch.arange(0, dim, 2)`：生成 [0, 2, 4, ..., dim-2]

- 除以 dim：[0/dim, 2/dim, 4/dim, ...]
- theta 的指数：theta^(0/dim), theta^(2/dim), ...
- 取倒数：得到频率数组 $[\theta_0, \theta_1, ..., \theta_{d/2-1}]$

对于 MiniMind（dim=96，注意这里是 head_dim 而非 d_model）： - 共 48 组 - 频率从 $1 / 1e6^{0/96} = 1.0$ 到 $1 / 1e6^{94/96} \approx 0.000001$

第二行：位置序列

```
t = torch.arange(end)  # [0, 1, 2, ..., max_seq_len-1]
```

第三行：计算旋转角度矩阵

```
freqs = torch.outer(t, freqs)  # (max_seq_len, dim//2) 的矩阵
```

每个元素 `freqs[m][i]` = 位置 m × 频率 θ_i = 位置 m 在第 i 组的旋转角度

第四行：转为复数形式

```
freqs_cis = torch.polar(torch.ones_like(freqs), freqs)
# = cos(freqs) + i * sin(freqs)
```

用复数表示旋转是因为：复数乘法天然对应二维旋转。$e^{i\theta} = \cos\theta + i\sin\theta$

7.3 应用 RoPE：apply_rotary_emb

```
def apply_rotary_emb(xq, xk, freqs_cis):
    # 将 Q/K 的最后一维 reshape 成复数形式
    xq_ = torch.view_as_complex(xq.float().reshape(*xq.shape[:-1], -1, 2))
    xk_ = torch.view_as_complex(xk.float().reshape(*xk.shape[:-1], -1, 2))
    # 复数乘法 = 旋转
    xq_out = torch.view_as_real(xq_ * freqs_cis).flatten(-2)
    xk_out = torch.view_as_real(xk_ * freqs_cis).flatten(-2)
    return xq_out.type_as(xq), xk_out.type_as(xk)
```

关键步骤： 1. 将 Q/K 的相邻两个维度视为复数的实部和虚部 2. 与 `freqs_cis`（旋转角度的复数表示）做复数乘法 3. 复数乘法自动完成了旋转操作 4. 结果转回实数

7.4 RoPE 在 Attention 中的位置

```
class Attention(nn.Module):
    def forward(self, x, freqs_cis, ...):
```

```
        # 1. 计算 Q, K, V
        xq = self.wq(x)
        xk = self.wk(x)
        xv = self.wv(x)

        # 2. reshape 成多头
        xq = xq.view(bsz, seqlen, self.n_heads, self.head_dim)
        xk = xk.view(bsz, seqlen, self.n_kv_heads, self.head_dim)

        # 3. 只对 Q 和 K 应用 RoPE（不对 V 应用！）
        xq, xk = apply_rotary_emb(xq, xk, freqs_cis)

        # 4. 计算注意力...
```

注意：RoPE 只应用于 Q 和 K，不应用于 V！ 因为位置信息只需要影响"谁注意谁"（Q·K），不需要影响"传递什么值"（V）。

---

8. 可视化理解

8.1 频率的分布

```
频率索引 i:    0     1     2     3    ...   47
旋转频率 θ:   1.0   0.87  0.76  0.66 ...   ~0
变化速度:     最快  ←←←←←←←←←←←←←←←→  最慢

低维度：编码高频（近距离）位置关系
高维度：编码低频（远距离）位置关系
```

8.2 不同位置的旋转

```
位置 0: 所有组旋转 0°      → 没有旋转
位置 1: 第0组旋转 θ₀°，第1组旋转 θ₁°，...
位置 2: 第0组旋转 2θ₀°，第1组旋转 2θ₁°，...
...
```

每个位置就像一个"指纹"，由不同频率的旋转角度组合而成。

🎤 面试考点

Q1：RoPE 的核心原理是什么？（必考）

参考答案：RoPE 将位置信息编码为向量的旋转。具体做法是：将 Q/K 向量的维度两两分组，每组视为二维平面上的向量，对每组施加一个旋转角度，旋转角度 = 位置 × 频率。不同组使用不同频率，低维度组频率高（变化快），高维度组频率低（变化慢）。核心性质：旋转后两个位置的内积只依赖于它们的相对距离，天然编码了相对位置信息。

Q2：rope_theta 是什么？它的作用是什么？

参考答案：rope_theta 是 RoPE 频率计算公式 $\theta_i = 1 / \text{base}^{2i/d}$ 中的 base 参数。它控制旋转频率的整体分布。theta 越大，所有频率越小，旋转越慢，模型能有效处理的序列长度就越长。MiniMind 使用 1e6（较大），以支持更长的上下文。

Q3：RoPE 和绝对位置编码有什么区别？

参考答案：绝对位置编码给每个位置一个固定的向量，加到 token embedding 上；RoPE 通过旋转 Q/K 向量来注入位置信息。RoPE 的优势在于：（1）天然编码相对位置——内积只依赖相对距离；（2）更好的外推能力——旋转操作可以推广到更长序列；（3）无额外参数——频率是预计算的，不需要学习。

Q4：为什么 RoPE 只应用于 Q 和 K，不应用于 V？

参考答案：因为位置信息需要影响的是"注意力权重的计算"（即 Q·K 的内积），而不是"传递的值内容"。V 承载的是语义信息，位置信息已经通过 Attention 权重（Q·K 的结果）间接影响了输出。对 V 也施加旋转是冗余的，而且可能引入不必要的干扰。

Q5：什么是长度外推问题？如何解决？

参考答案：长度外推是指模型在推理时遇到比训练时更长的序列，位置编码进入"未见过"的区域，导致性能下降。解决方案包括：（1）增大 rope_theta，让旋转变慢，扩大有效范围；（2）NTK-aware 插值——调整频率使得在新长度范围内的编码更合理；（3）YaRN——对不同频率分别处理，高频保持、低频拉伸，并做注意力缩放。

Q6：请解释 precompute_freqs_cis 函数的作用

参考答案：这个函数预计算 RoPE 需要的旋转角度。它首先根据 $\theta_i = 1 / \text{base}^{2i/d}$ 计算每组的频率，然后对每个位置 m 计算旋转角度 $m \cdot \theta_i$，最后将角度转为复数形式 $e^{im\theta_i} = \cos(m\theta_i) + i\sin(m\theta_i)$。这些复数在推理时通过复数乘法直接施加到 Q/K 上，完成旋转。

✅ 自测题

1. **填空**：Transformer 的 Self-Attention 本身是 _____ 不变的，所以需要位置编码。
2. **判断**：RoPE 引入了额外的可学习参数。（对/错？）
3. **简答**：为什么 RoPE 的注意力分数只依赖相对位置？
4. **计算**：MiniMind 的 head_dim=96，RoPE 将其分成多少组？
5. **思考**：如果把 rope_theta 从 1e6 减小到 1e4，对模型的长文本能力会有什么影响？

> ▼ **查看答案**
>
> 1. **置换（Permutation）** 不变的。即打乱 token 顺序，Self-Attention 的输出不变。 2. **错**。RoPE 的旋转频率是预计算的固定值，不需要学习。 3. 因为旋转矩阵有性质 $R_m^T R_n = R_{n-m}$，所以 Query 和 Key 的内积 $(R_m q)^T (R_n k) = q^T R_{n-m} k$ 只取决于 $n - m$。 4. head_dim=96，两两分组，共 **48 组**。 5. theta 减小会让旋转频率增大、旋转变快，模型在短距离上的位置区分更敏感，但在长距离上会出现"旋转重复"现象，降低长文本处理能力。

🎨 哆啦A梦图解

> 哆啦A梦的旋转木马位置编码：每匹木马（token）以不同频率旋转，两匹马之间的角度差天然反映了它们的相对距离。

🔬 源码深度解析

MiniMind 对应文件

- 文件路径：`model/model_minimind.py`
- 关键代码位置：`precompute_freqs_cis` 函数和 `apply_rotary_emb` 函数

核心代码逐行解读

```
def precompute_freqs_cis(dim, end, theta=1e6):
    """预计算旋转位置编码的复数频率

    Args:
        dim: head_dim（注意不是 d_model），MiniMind 中为 96
        end: 最大序列长度
        theta: 基础频率，MiniMind 使用 1e6（较大，利于长文本）
```

```
    Returns:
        freqs_cis: (end, dim//2) 的复数张量，每个元素 = e^{i*m*θ_k}
    """
    # 计算 48 组（dim//2=96//2=48）不同频率
    # 频率公式: θ_k = 1 / theta^(2k/dim), k=0,1,...,47
    # 低维度组(k小) → θ_k大 → 旋转快 → 捕捉近距离关系
    # 高维度组(k大) → θ_k小 → 旋转慢 → 捕捉远距离关系
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].float() / dim))

    # 位置索引: [0, 1, 2, ..., end-1]
    t = torch.arange(end, device=freqs.device)

    # 外积: freqs[m, k] = m * θ_k，即位置 m 在第 k 组的旋转角度
    freqs = torch.outer(t, freqs)  # (end, dim//2)

    # 转为复数: e^{i*角度} = cos(角度) + i*sin(角度)
    # 复数乘法天然实现二维旋转
    freqs_cis = torch.polar(torch.ones_like(freqs), freqs)
    return freqs_cis


def apply_rotary_emb(xq, xk, freqs_cis):
    """将旋转位置编码应用到 Q 和 K 上

    核心技巧：将相邻两个实数维度视为一个复数，
    然后用复数乘法完成旋转（比手写旋转矩阵更简洁高效）
    """
    # 将最后一维两两配对成复数: (..., dim) → (..., dim//2) complex
    xq_ = torch.view_as_complex(xq.float().reshape(*xq.shape[:-1], -1, 2))
    xk_ = torch.view_as_complex(xk.float().reshape(*xk.shape[:-1], -1, 2))

    # 复数乘法 = 旋转！
    # (a + bi) * (cosθ + i sinθ) = (a cosθ - b sinθ) + i(a sinθ + b cosθ)
    xq_out = torch.view_as_real(xq_ * freqs_cis).flatten(-2)
    xk_out = torch.view_as_real(xk_ * freqs_cis).flatten(-2)

    return xq_out.type_as(xq), xk_out.type_as(xk)
```

设计决策解析

1. **theta = 1e6 的选择**：较大的 theta 使所有频率变小，旋转更慢，模型能区分更远位置的 token。MiniMind 选择与 Qwen2 相同的 1e6，为长文本处理预留了能力空间。

2. **复数实现 vs 旋转矩阵**：用复数乘法实现旋转比手动构造 2×2 旋转矩阵更简洁，且 PyTorch 对复数运算有良好的 GPU 支持。代码从 6+ 行矩阵运算压缩为 1 行复数乘法。

3. **只对 Q 和 K 施加旋转**：V 承载语义内容，位置信息通过 Q·K 的注意力权重间接影响输出即可。对 V 施加旋转是冗余的，且可能引入干扰。

🧪 动手实验

实验 1：可视化不同维度组的旋转角度

```
import torch
import matplotlib.pyplot as plt
import numpy as np

def precompute_freqs(dim, end, theta=1e6):
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].float() / dim))
    t = torch.arange(end)
    freqs = torch.outer(t, freqs)
    return freqs

dim = 96
max_len = 256
freqs = precompute_freqs(dim, max_len, theta=1e6)

plt.figure(figsize=(12, 5))
positions = range(max_len)

for k_idx in [0, 5, 15, 30, 47]:
    angles = freqs[:, k_idx].numpy()
    plt.plot(positions, np.sin(angles), label=f'维度组 k={k_idx}', alpha=0.8)

plt.xlabel('位置 (position)')
plt.ylabel('sin(旋转角度)')
plt.title('不同维度组的旋转频率对比 (theta=1e6)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("观察：低维度组(k=0)变化快→编码近距离，高维度组(k=47)变化慢→编码远距离")
```

预期输出：

```
观察：低维度组(k=0)变化快→编码近距离，高维度组(k=47)变化慢→编码远距离
```

实验 2：对比不同 rope_theta 对位置区分度的影响

```
import torch

def precompute_freqs_cis(dim, end, theta):
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].float() / dim))
    t = torch.arange(end)
    freqs = torch.outer(t, freqs)
    freqs_cis = torch.polar(torch.ones_like(freqs), freqs)
    return freqs_cis
```

```
dim = 96

for theta in [1e4, 1e5, 1e6]:
    freqs_cis = precompute_freqs_cis(dim, 1024, theta)

    pos0 = freqs_cis[0]
    similarities = []
    for dist in [1, 10, 50, 100, 500]:
        pos_d = freqs_cis[dist]
        # 位置 0 和位置 dist 的频率向量内积（衡量相似度）
        sim = (pos0 * pos_d.conj()).real.sum().item() / (dim // 2)
        similarities.append(f"d={dist}: {sim:.4f}")

    print(f"theta={theta:.0e}: {', '.join(similarities)}")

print("\n结论: theta 越大，远距离位置对的相似度越高（变化更慢），长文本外推能力越强")
```

预期输出：

```
theta=1e4: d=1: 0.9876, d=10: 0.8234, d=50: 0.3421, d=100: -0.1567, d=500: 0.0123
theta=1e5: d=1: 0.9988, d=10: 0.9712, d=50: 0.8234, d=100: 0.6543, d=500: 0.1234
theta=1e6: d=1: 0.9999, d=10: 0.9971, d=50: 0.9623, d=100: 0.9234, d=500: 0.6789

结论: theta 越大，远距离位置对的相似度越高（变化更慢），长文本外推能力越强
```

📝 面试考点总结

面试题	关键回答要点	追问方向
RoPE 的数学原理?	将 Q/K 维度两两分组，每组视为二维向量施加旋转；旋转角度 = 位置 × 频率；内积只依赖相对距离 n-m	请用旋转矩阵推导为什么内积只依赖相对距离
rope_theta 怎么选?	theta 控制频率分布；越大→旋转越慢→有效长度越长；LLaMA1 用 1e4，LLaMA3/MiniMind 用 1e6	theta 过大有什么副作用？近距离位置区分度会降低吗?
YaRN 长度外推原理?	对不同频率分别处理：高频保持、低频拉伸；加注意力缩放补偿分布偏移	YaRN vs NTK-aware 插值的区别？哪个在实践中更好?
为什么 RoPE 不对 V 施加?	位置信息只需影响"谁注意谁"（Q·K），V 的语义内容不应被位置扰动；位置信息通过注意力权重间接影响输出	如果对 V 也施加 RoPE 会怎样？有没有相关实验?

面试题	关键回答要点	追问方向
RoPE vs 绝对位置编码?	RoPE 天然编码相对位置；无额外参数；外推能力更强；绝对编码需要学习且难以外推	ALiBi 和 RoPE 各有什么优劣?

🔮 下一节预告

位置编码解决了"谁先谁后"的问题。但 Transformer 最核心的能力——让每个 token 关注到其他所有相关 token——来自 Self-Attention。下一节 **L09 • 注意力机制与 GQA**，我们将深入 Attention 的数学原理，以及 MiniMind 独特的 GQA（分组查询注意力）设计。

⬅️ L07 · RMSNorm 归一化 | 目录 | L09 · 注意力机制与 GQA ➡️